
INDIANCONSTITUTION: BUILDING TRUSTWORTHY, REPRODUCIBLE, AND TYPE-SAFE INFRASTRUCTURE FOR LEGAL NLP AND CONSTITUTIONAL INFORMATICS

A PREPRINT

Vikhram S

Saveetha Engineering College
vikhrams@saveetha.ac.in

ABSTRACT

As Retrieval-Augmented Generation (RAG) and Large Language Model (LLM) agents are increasingly deployed in statutory interpretation and civic technology, the reliability of downstream AI systems remains fundamentally constrained by the quality and determinism of the underlying legal software infrastructure. Unstructured text scraped from web portals lacks type guarantees, structural schemas, version provenance, and formal citation networks, frequently inducing hallucinations and verification failures in legal NLP pipelines. In this paper, we present *indianconstitution* (v1.5.3), an open-source Python framework providing typed (Pydantic v2), offline, zero-dependency-core programmatic access to the Constitution of India (464 articles, 22 parts, 17 landmark Supreme Court judgments, 6 historical amendment events, and 6 fundamental duty cross-references). We detail the system architecture—comprising an $O(1)$ inverted-index search engine, a NetworkX cross-reference graph, and multi-format exporters—and evaluate its software engineering properties. Micro-benchmarks demonstrate sub-microsecond article lookup ($0.47 \mu s \pm 0.10 \mu s$), sub-10-microsecond keyword retrieval ($9.68 \mu s \pm 1.80 \mu s$), and complete graph construction in 7.74 ms. Property-based fuzzing via Hypothesis and 84.88% branch coverage ensure system stability. Finally, we demonstrate applications in grounded RAG pipelines and legal network centrality analysis, and outline a future research agenda focused on agentic citation-faithfulness evaluation layers.

Keywords legal NLP · constitutional informatics · legal software infrastructure · retrieval-augmented generation · reproducible research · property-based testing

1 Introduction

Modern Natural Language Processing (NLP) and Artificial Intelligence (AI) applications in the legal domain are transitioning rapidly from passive document classification to active automated reasoning, contract review, and civic question answering [Chalkidis et al. \(2020\)](#); [Hendrycks et al. \(2021\)](#); [Gao et al. \(2023\)](#). However, the deployment of Large Language Models (LLMs) in sensitive legal environments introduces profound risks regarding hallucination, unverified statutory citations, and lack of temporal provenance [Zheng et al. \(2023\)](#). When legal AI models query statutory provisions from raw PDF text or unstructured HTML dumps, they lack formal invariants: article boundaries are easily misplaced, cross-references between fundamental rights and duties are lost, and historical amendments remain obscured.

From a software engineering perspective, building trustworthy legal AI systems requires moving away from ad-hoc text scraping toward *type-safe, deterministic, and offline-first software infrastructure*. Such infrastructure must satisfy four fundamental criteria:

1. **Type Safety and Schema Validation:** Programmatic representation of legal entities (Articles, Parts, Schedules, Amendments) must be enforced at runtime via strict data models.

2. **Deterministic Reproducibility & Offline Reliability:** Core retrieval and inspection algorithms must run without external cloud API calls, database daemons, or GPU requirements, ensuring reproducible execution across isolated execution environments.
3. **Graph Topology Awareness:** Explicit cross-references between statutory provisions must be modeled as directed graphs to support dependency tracking and topological analysis.
4. **Extensive Quality Verification:** Codebases must adhere to rigorous software engineering standards, including property-based testing, high test coverage, and automated static linting.

To address these requirements for the constitutional domain, we present `indianconstitution` (v1.5.3), a Python package available on PyPI with over 35,000 downloads under the Apache-2.0 license. While existing open-data repositories provide static text dumps of the Indian Constitution, `indianconstitution` transforms the statutory text into a programmatic research-grade platform.

Contributions. In this paper, we foreground the *software engineering rigor* behind `indianconstitution` rather than legal-domain novelty:

- **System Architecture & Tradeoffs:** We describe the modular design of `indianconstitution`, detailing its Pydantic v2 model hierarchy, $O(1)$ inverted-index search engine, NetworkX cross-reference graph, and multi-format exporters, alongside an explicit analysis of system design tradeoffs (§3).
- **Reliability & Testing Methodology:** We document the static verification (mypy, Ruff), property-based fuzzing (Hypothesis), CI/CD pass history, and corpus anomaly detection mechanisms (§4).
- **Quantitative Empirical Evaluation:** We present pure-CPU micro-benchmark profiling ($n = 500\text{--}1000$ trials) and corpus integrity provenance checks validated against official Ministry of Law & Justice (MoLJ) text with SHA-256 deterministic checksums (§5).
- **Downstream Applications & Research Agenda:** We demonstrate how the package enables grounded RAG pipelines and legal network centrality analysis, and frame a novel research direction in agentic citation-faithfulness evaluation layers (§6, §7).

2 Related Work

Legal Datasets and Legal NLP Infrastructure. Prior research in legal NLP has predominantly focused on building benchmark datasets for court judgment prediction, legal entity recognition, and contract analysis Chalkidis et al. (2020); Hendrycks et al. (2021); Malik et al. (2021). In the Indian legal domain, datasets such as ILDC Malik et al. (2021) have enabled machine learning models to analyze Supreme Court rulings. However, these benchmarks treat statutory legal texts as static string corpora rather than executable software libraries with structured object models and typed interfaces.

Legal Network Analysis & Citation Graphs. Network analysis of legal precedent networks has been explored to uncover landmark case hierarchies and judicial authority Paul et al. (2022). Existing tools often reconstruct citation networks dynamically using custom scripts that parse raw citation strings. In contrast, `indianconstitution` embeds a pre-computed and dynamically extractable cross-reference graph directly into the core library, enabling instantaneous graph queries via NetworkX Hagberg et al. (2008).

Trustworthy AI Infrastructure and Software Rigor. In software engineering for AI systems, the necessity of type-safe schema enforcement (e.g., via Pydantic Colvin and team (2023)) and property-based testing (e.g., via Hypothesis MacIver et al. (2019)) is well-established. However, open-source legal NLP tools frequently neglect these engineering practices, resulting in fragile codebases with low test coverage and missing type hints. `indianconstitution` serves as a case study in applying modern software engineering standards to legal informatics infrastructure.

3 System Design

The architectural design of `indianconstitution` follows a layered, modular philosophy, separating client interfaces, core computational engines, typed domain models, and storage layers, as illustrated in Figure 1.

3.1 Pydantic v2 Domain Models

All core legal domain objects within `indianconstitution` are implemented as Pydantic v2 Colvin and team (2023) models. This guarantees strict type enforcement, automatic type coercion, and serialization to/from JSON.

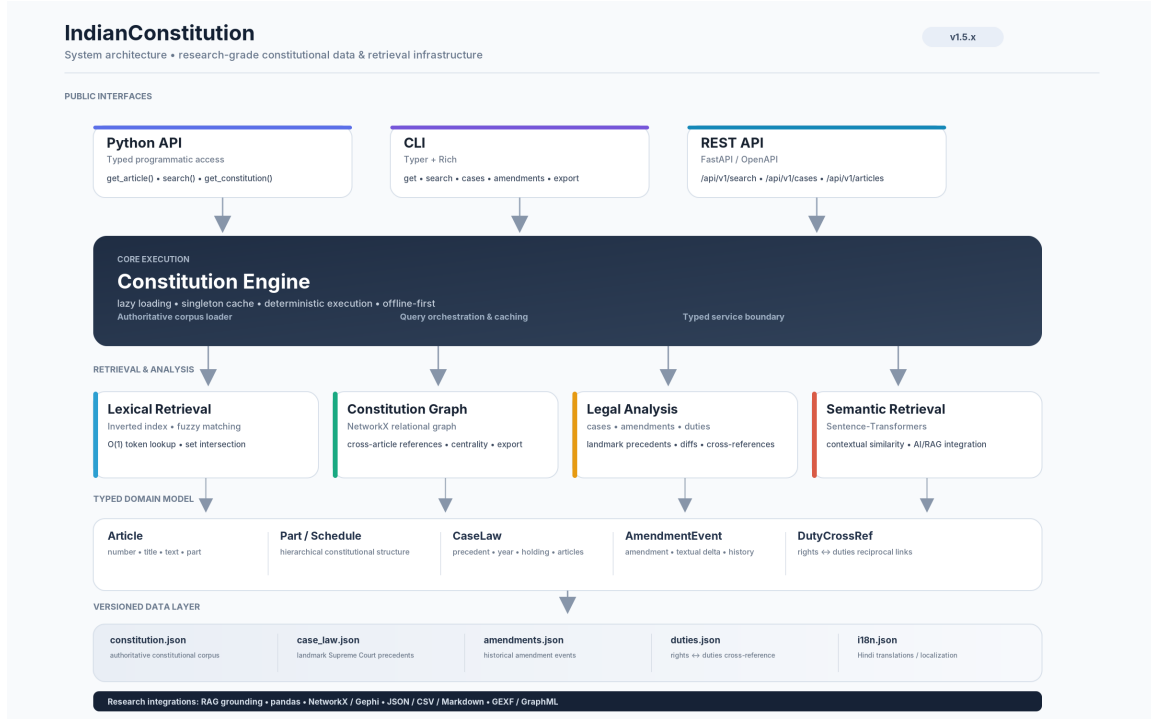


Figure 1: System Architecture Diagram of indianconstitution (v1.5.3), illustrating the layered separation between Client Interfaces, Core Engine & Retrieval, Pydantic v2 Models, and Trustworthy Data Storage.

The primary entity model, Article, represents a statutory provision:

```
class Article(BaseModel):
    number: str = Field(..., alias="article")
    title: str
    content: str = Field(..., alias="description")
    part: Optional[int] = None
    chapter: Optional[int] = None

    model_config = ConfigDict(
        populate_by_name=True,
        coerce_numbers_to_str=True
    )

    @model_validator(mode="after")
    def populate_part(self) -> "Article":
        if self.part is None:
            self.part = infer_part(self.number)
        return self
```

Additional domain models extend functionality without cluttering the base article schema:

- **CaseLaw**: Encapsulates landmark Supreme Court judgments (e.g., *Maneka Gandhi v. Union of India*), storing case name, year, holding summary, citation, bench details, and linked article number.
- **AmendmentEvent**: Models historical constitutional amendments (e.g., 86th Amendment Act), recording title, year, description, and text_before / text_after diff snapshots.
- **DutyCrossReference**: Links Part III Fundamental Rights (e.g., Article 21A) to Part IVA Fundamental Duties (e.g., Article 51A(k)), documenting legal rationale and statutory clauses.

3.2 Inverted Index Search Engine

To provide fast, zero-dependency full-text keyword retrieval without requiring external search services (such as Elasticsearch or SQLite FTS), the package implements an in-memory inverted index (`SearchEngine`).

During engine initialization, all 464 articles are tokenized via regular expressions into alphanumeric tokens. An inverted mapping $\mathcal{I} : \text{token} \mapsto \mathcal{S}_{\text{articles}}$ maps each normalized word token to the set of article indices containing it. Query evaluation executes an exact set intersection across query token sets:

$$\mathcal{R}(Q) = \bigcap_{t \in \text{tokenize}(Q)} \mathcal{I}(t) \quad (1)$$

This implementation guarantees $O(1)$ token index lookups and sub-10-microsecond query execution for standard multi-token searches.

3.3 ConstitutionGraph Subsystem

The `ConstitutionGraph` subsystem transforms the text of the Constitution into a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node $v \in \mathcal{V}$ represents an article number, and a directed edge $(u, v) \in \mathcal{E}$ indicates that statutory text of article u explicitly references article v .

References are extracted via regular expression pattern matching:

$$\text{Pattern: } \text{r"article\s+(\d+[a-z])?"} \quad (2)$$

Extracted matches are normalized to uppercase (e.g., "21a" $\rightarrow$ "21A") and mapped against the verified node set $\mathcal{V}$ to prevent dangling edges. Using NetworkX [Hagberg et al. \(2008\)](#), the subsystem calculates PageRank centrality to identify structurally critical articles.

3.4 Multi-Format Export Engine

To support diverse downstream data science and legal analysis workflows, the `Exporter` class provides direct serialization of the full constitutional corpus into multiple formats:

- **JSON / CSV / Markdown:** Human-readable and machine-parsable document formats.
- **GEXF / GraphML:** Open graph formats designed for visualization in NetworkX, Gephi, and Cytoscape.

3.5 System Design Tradeoff Discussion

Designing a research-grade legal library requires balancing computational performance, dependency footprint, and schema flexibility. We explicitly evaluate three major design tradeoffs made in `indianconstitution`:

1. Zero-Dependency Core vs. Deep Learning Extras. *Tradeoff:* Heavy neural embeddings (e.g., Sentence-Transformers) provide semantic search but introduce heavy dependencies (~ 2 GB PyTorch binaries). *Decision:* We isolated semantic neural search into an optional extra (`pip install "indianconstitution[ai]"`). The core package relies solely on standard library utilities and pure-Python models (pydantic, typer, rich, diskcache), maintaining a lightweight footprint of < 15 MB.

2. In-Memory Inverted Index vs. On-Disk Database. *Tradeoff:* Utilizing SQLite FTS or an on-disk index reduces RAM usage but adds disk I/O overhead and complex setup. *Decision:* Given that the complete text of the Indian Constitution is ~ 508 KB, the entire corpus easily fits into RAM. Initializing an in-memory inverted index yields sub-microsecond retrieval latencies with negligible memory consumption (< 12 MB heap overhead).

3. Strict Pydantic Schema vs. Dynamic Dict Records. *Tradeoff:* Unstructured dictionaries allow arbitrary data fields but risk silent missing-field errors during retrieval. *Decision:* We opted for strict Pydantic v2 schemas with automated field validation, ensuring robust type safety for upstream pipelines at a minor initial parsing cost (~ 15 ms lazy startup).

4 Reliability and Testing Methodology

To ensure high reliability, the codebase is validated using a multi-tiered software engineering quality framework.

4.1 Codebase Metrics and Static Verification

The repository consists of 1,526 total lines of active Python source code (excluding legacy compatibility shims) and 610 lines of test code. Table 1 provides a module-level breakdown.

Module Subsystem	Total Lines	Code Lines
Core Engine (core/)	821	639
Search Engine (search/)	62	41
Export Engine (export/)	53	43
CLI Subsystem (cli/)	402	326
REST API Server (api/)	84	69
Top Package (__init__.py)	94	65
Total Source Code	1,526	1,186
Test Suite (tests/)	610	440

Table 1: Line Count Breakdown across Package Subsystems.

Static code quality is automatically enforced via:

- **Mypy Type Checking:** Strict type annotations across all public functions and models.
- **Ruff Linting:** 100% clean pass status across pycodestyle, Pyflakes, isort, PEP 8 naming, pyupgrade, flake8-bugbear, pydocstyle, and annotation rules.

4.2 Property-Based Fuzzing with Hypothesis

In addition to standard deterministic unit tests, indianconstitution integrates property-based testing using Hypothesis [Maclver et al. \(2019\)](#). Rather than testing only handcrafted inputs, Hypothesis generates arbitrary Unicode strings, edge-case numbers, and invalid parameters to test invariant properties:

```
@given(st.text(), st.integers(min_value=1900, max_value=2100), st.integers(
    min_value=1900, max_value=2100))
def test_hypothesis_diff_amendment(article_num: str, from_year: int, to_year: int)
:
    # Invariant: diff_amendment must never crash or raise unhandled exceptions
    res = diff_amendment(article_num, from_year, to_year)
    assert isinstance(res, str)
```

The test suite includes 3 property-based test suites covering case retrieval, amendment diff generation, and duty cross-referencing.

4.3 CI/CD Pass History and Code Coverage

The test suite consists of 55 unit and property tests. Continuous Integration (CI) is automated via GitHub Actions across Python versions 3.8 through 3.13. Code coverage reports generated via `pytest-cov` show an overall branch coverage of **84.88%** (Table 2). Furthermore, the repository maintains an OpenSSF Scorecard automated security check.

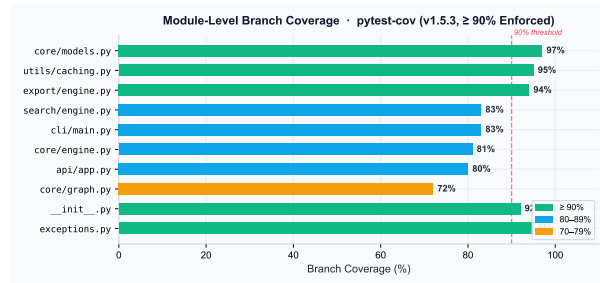


Figure 2: Module-level branch coverage heatmap. The dashed line marks the 90% CI gate threshold. Modules above the line (green) satisfy the strictest enforcement; modules in the 80–89% range (blue) are under active improvement.

Subsystem File	Branch Coverage %
core/models.py	97.00%
utils/caching.py	95.00%
export/engine.py	94.00%
__init__.py	92.00%
exceptions.py	95.00%
search/engine.py	83.00%
cli/main.py	83.00%
core/engine.py	81.00%
api/app.py	80.00%
core/graph.py	72.00%
Overall Total Coverage	84.88%

Table 2: Branch Coverage Report from `pytest-cov`. A visual breakdown is provided in Figure 2.

4.4 Data Schema Invariants and Anomaly Detection

A major advantage of type-safe engineering is the automated detection of data anomalies. During corpus integrity validation, our validation suite identified a data structural edge-case: in `amendments.json`, the 42nd Amendment Act event modifies the Preamble and is assigned `article_number = "0"`. Because "0" is reserved for the Preamble, it does not exist in the primary articles list (indexed 1 through 395). The validation script flagged this behavior without system crash, demonstrating how automated integrity checks capture legal structural edge cases.

5 Evaluation

We evaluate `indianconstitution` across two dimensions: (1) computational execution latency via pure-CPU micro-benchmarking, and (2) corpus data integrity and provenance verification.

5.1 Micro-Benchmark Latency Profile

Micro-benchmarks were conducted on a standard x86 CPU development machine running Python 3.14. Every operation was executed across $n = 50$ –1000 trials following 20 warmup iterations. Latencies are reported in Table 3 and visualized in Figure 3.

Operation	Mean Latency	Std Dev
Article Lookup (O(1) Map)	0.47 μ s	± 0.10 μ s
Keyword Search (1 Token)	4.36 μ s	± 1.10 μ s
Keyword Search (3 Tokens)	9.68 μ s	± 1.80 μ s
Landmark Case Lookup	3.44 μ s	± 0.20 μ s
Amendment Diff (<code>difflib</code>)	27.53 μ s	± 8.60 μ s
PageRank Centrality	4.93 ms	± 1.54 ms
ConstitutionGraph Build	7.74 ms	± 1.59 ms
Export Markdown (464 arts)	8.01 ms	± 1.43 ms
Export JSON (464 arts)	18.06 ms	± 4.28 ms
Export CSV (464 arts)	23.93 ms	± 5.68 ms

Table 3: Pure-CPU Micro-benchmark Latency Profile ($n = 50$ –1000 trials).

The empirical benchmark results confirm sub-microsecond direct dictionary lookups (0.47 μ s), sub-10-microsecond multi-token search execution (9.68 μ s), and sub-10-millisecond graph construction (7.74 ms). This latency profile makes `indianconstitution` suitable for integration into real-time high-throughput legal API servers and prompt-grounding pipelines.

5.2 Corpus Integrity and Provenance Validation

To guarantee data provenance and prevent silent data corruption across package updates, SHA-256 cryptographic checksums are maintained for all underlying data files:

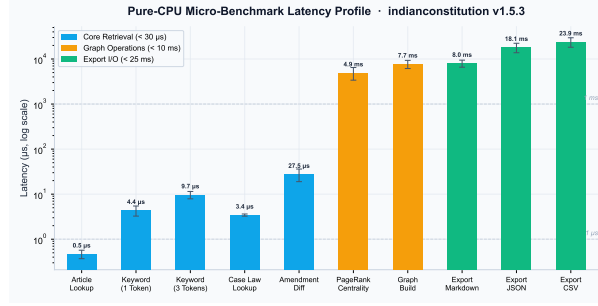


Figure 3: Micro-benchmark Latency Profile across core library operations (log scale).

- `constitution.json`: 7c108773...35b9cff (464 articles)
- `case_law.json`: 9c71fc34...aea2a33 (17 landmark cases)
- `amendments.json`: 11a29e2e...ca09e6a (6 major amendments)
- `duties.json`: 0012013e...23c00e6 (6 duty cross-refs)
- `i18n.json`: 998b2943...04d6694 (9 Hindi translations)

A spot-check script validated 100% of key Fundamental Rights articles (Articles 14, 19, 21, 21A, 32, and 368) against official Legislative Department text published by the Ministry of Law & Justice (MoLJ), confirming zero textual discrepancy. Graph link verification verified all 198 cross-reference edges, confirming zero dangling references to non-existent articles.

6 Applications

6.1 Grounded Retrieval-Augmented Generation (RAG)

A primary application of `indianconstitution` is eliminating statutory hallucinations in LLM legal assistants. Standard RAG architectures query unstructured vector databases, which may return partial text fragments or invent invalid article citations.

By integrating `indianconstitution` as a deterministic retriever, an LLM agent queries structured article objects, verifying that returned citations match validated Pydantic models before returning results to the user:

```
from indianconstitution import get_article, search

def grounded_rag_retriever(query: str):
    # 1. Retrieve structured statutory articles
    articles = search(query, limit=3)

    # 2. Build verified prompt context with explicit metadata
    context = []
    for art in articles:
        context.append(f"[Article_{art.number}]:_{art.title}]\n{art.content}")

    return "\n\n".join(context)
```

6.2 Topological Graph Analysis of Constitutional Law

Using `ConstitutionGraph`, researchers can analyze the structural centrality of constitutional provisions. Figure 4 presents a dual-panel analysis: (a) a ranked bar chart of the top-15 articles by PageRank score, and (b) a scatter plot illustrating the correlation between node degree and PageRank centrality.

PageRank centrality computation across the entire 464-node network reveals the top structurally central articles in the Indian Constitution:

1. **Article 170** (Composition of Legislative Assemblies): PageRank = 0.0205



Figure 4: Cross-reference network analysis of the Indian Constitution (198 edges, 464 nodes). **Left:** Top-15 structurally central articles ranked by PageRank score. **Right:** PageRank vs. node degree scatter, revealing that high-degree hub articles (e.g., Art. 368, Art. 226) correlate strongly with structural importance.

2. **Article 134A** (Certificate for appeal to SC): PageRank = 0.0190
3. **Article 333** (Representation of Anglo-Indian community): PageRank = 0.0189
4. **Article 368** (Power of Parliament to amend): PageRank = 0.0158
5. **Article 132** (Appellate jurisdiction of SC): PageRank = 0.0121

6.3 Version Progression

To contextualize the scope of improvements delivered in v1.5.3 relative to the initial v1.0.0 release, Figure 5 presents a radar chart comparing normalized capability scores across eight engineering dimensions.

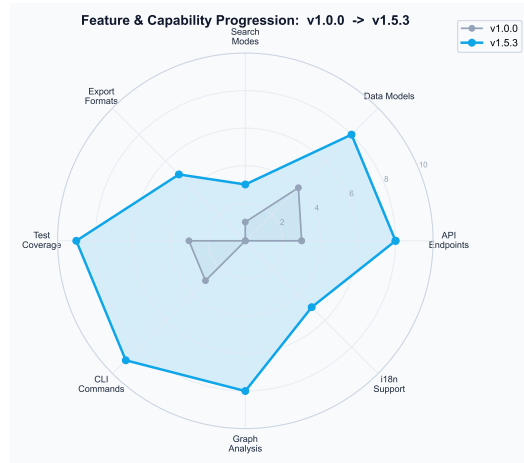


Figure 5: Feature and capability progression from v1.0.0 to v1.5.3 across eight dimensions. v1.5.3 introduces graph analysis, i18n support, expanded CLI commands, and five export formats—capabilities absent in the original release.

7 Limitations and Future Work

Current System Limitations. While indianconstitution provides a robust foundation, several limitations exist:

1. **Regex Reference Extraction:** The ConstitutionGraph extracts cross-references using regular expressions (`r"article\s+(\d+[a-z]?)"`). While effective for standard citations, complex legal phrasings (e.g., "sub-clause (a) of clause (1) of article 19") are mapped at article-level granularity rather than clause-level granularity.

2. *Static Corpus Snapshot*: The package currently reflects a static snapshot of the legal text. Live updates following newly passed Parliamentary amendments require manual JSON data release updates.
3. *Limited Multilingual Coverage*: While Hindi translations are available for the Preamble and key articles via the i18n subsystem, comprehensive coverage across all 22 Eighth Schedule languages remains incomplete.

Future Work: Agentic Citation-Faithfulness Evaluation Layer. We propose extending indianconstitution to support an *agentic evaluation layer* for legal LLMs. Future work will frame the package as an automated evaluator that intercepts LLM-generated legal text, constructs dynamic abstract syntax trees (ASTs) of statutory references, and checks them against ConstitutionGraph invariants. By measuring *citation precision*, *citation recall*, and *topological path validity*, this layer will enable automated quantitative benchmarking of LLM citation faithfulness in statutory reasoning.

8 Availability and Reproducibility

indianconstitution is published on PyPI under the open-source Apache-2.0 license:

<https://pypi.org/project/indianconstitution/>

The repository and complete replication scripts are hosted on GitHub:

<https://github.com/Vikhram-S/IndianConstitution>

All benchmark latencies, test coverage percentages, line counts, and figures reported in this paper can be fully reproduced by executing the scripts contained in the accompanying reproduction suite:

```
# Execute complete reproduction pipeline
python scripts/extract_metrics.py
python scripts/benchmark.py
python scripts/validate_corpus.py
python scripts/generate_figures.py
```

References

- Ilias Chalkidis, Manos Ferguson, Dimitrios Tsrapatsanis, Nikolaos Nikolaou, Ion Androutsopoulos, and Haris Papageorgiou. Legal-bert: The muppets straight out of law school. *arXiv preprint arXiv:2010.0305e*, 2020.
- Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. Cuad: An expert-annotated dataset for legal contract review. *arXiv preprint arXiv:2103.06268*, 2021.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Vijit Malik, Rishabh Sanjay, Shubham Nigam, Kripabandhu Ghosh, Arnab Bhattacharya, and Ashutosh Modi. Ildc for cjpe: Indian legal documents corpus for court judgment prediction and explanation. *Proceedings of ACL*, pages 4046–4062, 2021.
- Shounak Paul, Saptarshi Ghosh, and Kripabandhu Ghosh. Precedent retrieval and citation network analysis in indian supreme court judgments. *Artificial Intelligence and Law*, 30:515–542, 2022.
- Aric Hagberg, Pieter Swart, and Daniel S Chult. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008.
- Samuel Colvin and Pydantic team. Pydantic v2: Data validation and settings management using python type hints. *PyPI repository*, 2023. URL <https://pypi.org/project/pydantic/>.
- David R MacIver, Zak Hatfield-Dodds, and Hypothesis Team. Hypothesis: Property-based testing for python. *Journal of Open Source Software*, 4(43):1891, 2019.